

TP : Détection de voies routières

Vision par ordinateur avec OpenCV en C++

Maryam Boufous

1. Objectifs du TP

L'objectif de ce TP est d'implémenter un système complet de détection de voies routières en utilisant les techniques de vision par ordinateur avec OpenCV en C++. Vous implémenterez étape par étape les différentes phases du traitement d'image.

Compétences évaluées :

- Compréhension des espaces de couleur HSV
- Transformation de perspective (*bird eye view*)
- Détection de contours et seuillage (algorithme de Canny)
- Méthode des fenêtres glissantes (*sliding windows*)
- Ajustement polynomial et calcul de courbure

2. Vue d'ensemble du pipeline

Le système de détection de voies suit un pipeline séquentiel de 10 étapes. Chaque étape transforme l'image pour extraire progressivement l'information sur les voies.

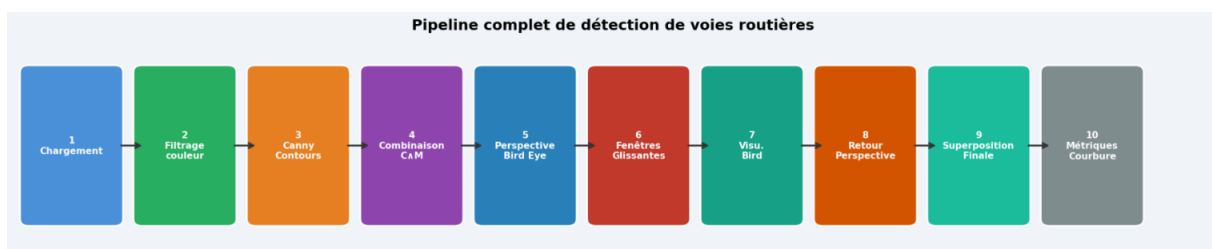


FIGURE 1 – Vue d'ensemble du pipeline complet de détection de voies routières

3. Concepts fondamentaux et définitions

3.1 Espace de couleur HSV

Définition — Espace HSV (Hue, Saturation, Value)

L'espace HSV décompose une couleur en trois composantes indépendantes :

- **H (Teinte / Hue)** : la couleur pure, représentée par un angle. Dans OpenCV, la plage est **0–180** (au lieu de 0–360° en théorie) afin de tenir sur un octet.
- **S (Saturation)** : la pureté ou intensité de la couleur, de 0 (gris pur) à 255 (couleur saturée). Une saturation faible correspond à du blanc ou du gris.
- **V (Valeur / Luminosité)** : la brillance, de 0 (noir) à 255 (très lumineux).

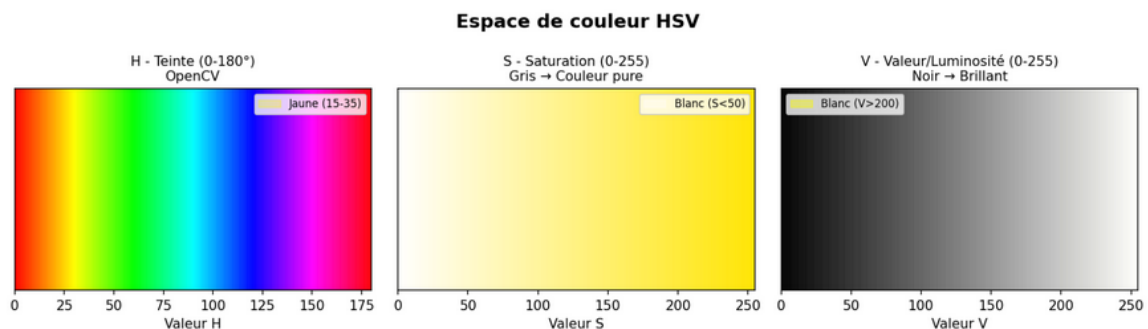


FIGURE 2 – Les trois composantes HSV et les plages utilisées pour détecter le blanc et le jaune

Pourquoi HSV plutôt que RGB ?

En espace RGB, les variations d'éclairage (ombre, soleil, nuit) modifient *simultanément* les trois canaux R, G et B, rendant difficile la définition d'un seuil de couleur stable. En HSV, la teinte H reste constante quelle que soit la luminosité : seul V change. On peut donc détecter le **jaune** (H entre 15 et 35) ou le **blanc** ($S < 50$, $V > 200$) de façon robuste.

Valeurs HSV typiques pour les marquages routiers :

Couleur	H (Teinte)	S (Saturation)	V (Valeur)
Blanc	0 – 180	0 – 50	200 – 255
Jaune	15 – 35	60 – 255	100 – 255

3.2 Algorithme de Canny — Détection de contours

Définition — Algorithme de Canny (1986)

L'algorithme de Canny détecte les **gradients d'intensité** dans l'image, c'est-à-dire les zones où la luminosité change brusquement (= les bords des objets). Il se compose de 4 étapes internes :

1. **Lissage gaussien** : suppression du bruit par convolution avec un noyau gaussien.
2. **Calcul du gradient** : détection de la magnitude et direction du gradient en chaque pixel (filtres de Sobel).
3. **Suppression des non-maxima** : on ne conserve que les pixels qui sont des maxima locaux dans la direction du gradient (amincissement des contours).
4. **Double seuillage par hystérésis** :
 - Pixel avec gradient $>$ seuil haut : **contour certain** (conservé).
 - Pixel avec gradient $<$ seuil bas : **rejeté** (bruit).
 - Pixel entre les deux seuils : conservé **seulement** s'il est connecté à un contour certain.

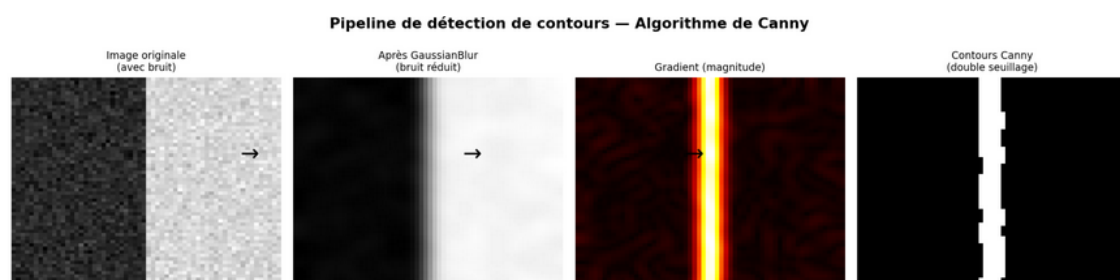


FIGURE 3 – Pipeline interne de l'algorithme de Canny : du bruitage aux contours nets

Choix des seuils Canny

Les seuils 50 et 150 sont des valeurs de départ courantes. Un rapport 1 :3 est recommandé. Si les seuils sont trop bas, du bruit est détecté ; trop hauts, des contours réels sont perdus.

3.3 Transformation de perspective

Définition — Transformation homographique (Perspective Transform)

Une **transformation de perspective** (ou homographie) mappe 4 points sources vers 4 points destination dans un plan différent. Elle modifie l'angle de vue apparent de la scène :

- Vue caméra inclinée : les voies convergent vers un point de fuite (effet perspective).
- Vue de dessus (*bird eye view*) : les voies sont parallèles et la distance est proportionnelle à la position verticale dans l'image.

Elle est représentée par une matrice 3×3 et s'applique aux coordonnées homogènes.

Transformation de perspective — Vue caméra → Vue de dessus (Bird Eye View)

Vue caméra (perspective)
Lignes convergentes

Vue de dessus (bird eye)
Lignes parallèles



FIGURE 4 – Passage de la vue caméra (trapèze rouge) à la vue bird eye (rectangle bleu)

Différence transformation affine vs perspective :

- **Affine** (matrice 2×3) : préserve les lignes parallèles. Utilisée pour rotation, translation, mise à l'échelle.
- **Perspective** (matrice 3×3) : ne préserve *pas* le parallélisme. Utilisée pour simuler un changement de point de vue : c'est notre cas ici.

3.4 Méthode des fenêtres glissantes

Définition — Sliding Windows — Fenêtres glissantes

La méthode des fenêtres glissantes divise l'image verticalement en N bandes horizontales. Pour chaque bande (du bas vers le haut) :

1. Une fenêtre rectangulaire est placée autour de la position courante de la voie.
2. On collecte tous les pixels blancs à l'intérieur de cette fenêtre.
3. Si le nombre de pixels collectés dépasse un seuil (`min_pixels`), on recentre la prochaine fenêtre sur la moyenne de leurs positions en x .

La position de départ (en bas) est déterminée par les pics de l'**histogramme** de la moitié basse de l'image.

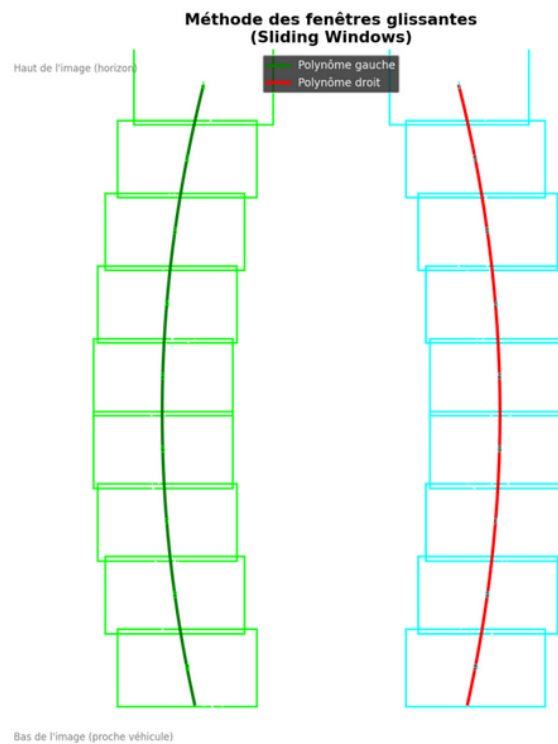


FIGURE 5 – Fenêtres glissantes (vert = voie gauche, cyan = voie droite) et polynômes ajustés

3.5 Ajustement polynomial et rayon de courbure

Définition — Ajustement polynomial — Moindres carrés

On modélise chaque voie par un polynôme du second degré :

$$x = Ay^2 + By + C$$

où x est la position horizontale et y la position verticale dans l'image (on ajuste x en fonction de y et non l'inverse, car les voies peuvent être presque verticales).

Les coefficients A , B , C sont trouvés par la **méthode des moindres carrés** : on résout le système sur-déterminé $\mathbf{A} \mathbf{c} = \mathbf{b}$ (via décomposition SVD dans OpenCV avec `solve(..., DECOMP_SVD)`).

Définition — Rayon de courbure

Pour une courbe $x = f(y)$, le rayon de courbure en un point est :

$$R = \frac{\left[1 + \left(\frac{dx}{dy}\right)^2\right]^{3/2}}{\left|\frac{d^2x}{dy^2}\right|}$$

Pour notre parabole $x = Ay^2 + By + C$: $\frac{dx}{dy} = 2Ay + B$ et $\frac{d^2x}{dy^2} = 2A$, d'où :

$$R = \frac{[1 + (2Ay + B)^2]^{3/2}}{|2A|}$$

Un grand R indique une route presque droite ; un petit R un virage serré.

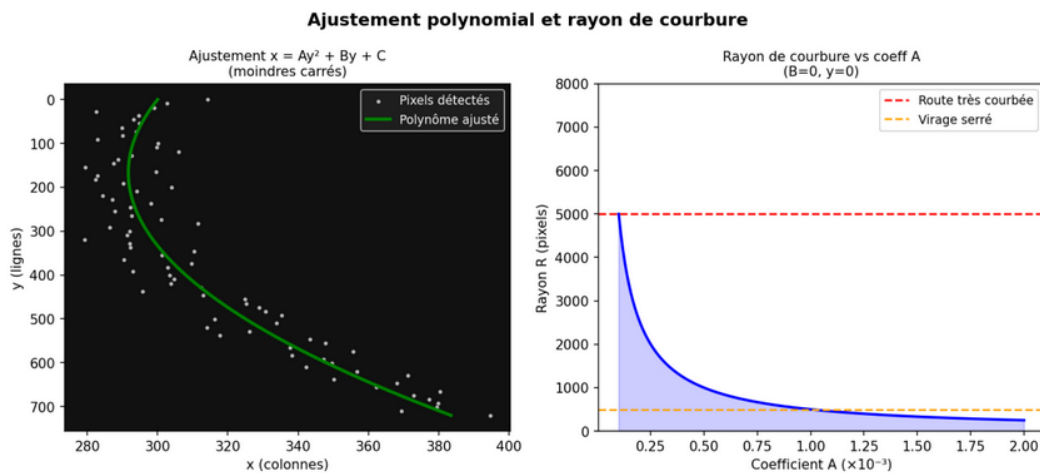


FIGURE 6 – Ajustement polynomial sur les pixels de voie (gauche) et variation du rayon de courbure (droite)

4. Fonctions OpenCV essentielles

Fonction	Utilisation
<code>cvtColor()</code>	Conversion d'espaces : BGR $\rightarrow$ HSV, RGB, GRAY
<code>inRange()</code>	Masquage binaire par seuils HSV
<code>GaussianBlur()</code>	Réduction du bruit avant Canny
<code>Canny()</code>	Détection de contours par double seuillage
<code>bitwise_or()</code> / <code>bitwise_and()</code>	Combinaison logique de masques binaires
<code>getPerspectiveTransform()</code>	Calcul de la matrice de transformation 3×3
<code>warpPerspective()</code>	Application de la transformation de perspective
<code>findNonZero()</code>	Extraction de la liste de pixels non nuls
<code>reduce()</code>	Calcul d'histogramme (somme par colonne)
<code>minMaxLoc()</code>	Localisation du pic dans l'histogramme
<code>solve()</code>	Résolution par moindres carrés (SVD) pour le polynôme
<code>polylines()</code>	Dessin des lignes de voie détectées
<code>fillPoly()</code>	Remplissage de la zone entre les voies
<code>addWeighted()</code>	Superposition semi-transparente sur l'image originale
<code>putText()</code>	Affichage des métriques (rayon, décalage)

5. Étapes détaillées du pipeline

5.1 Étape 1 : Chargement et prétraitement

Principe

Charger l'image depuis le disque et la convertir dans un format adapté au traitement. OpenCV charge en BGR par défaut ; on convertit en RGB pour l'affichage final correct.

```

1 Mat image = imread("route.jpg");
2 if (image.empty()) {
3     cerr << "Erreur: Impossible de charger l'image" << endl;
4     return -1;
5 }
6 Mat image_rgb;
7 cvtColor(image, image_rgb, COLOR_BGR2RGB);

```

Listing 1 – Chargement de l'image

5.2 Étape 2 : Filtrage par couleur

Principe

L'espace HSV permet d'isoler les marquages blancs et jaunes indépendamment des conditions d'éclairage.

```

1 Mat hsv;
2 cvtColor(image, hsv, COLOR_BGR2HSV);
3

```

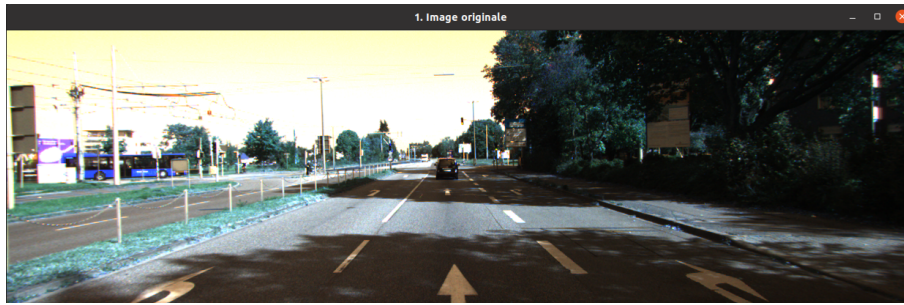


FIGURE 7 – Étape 1 — Image originale chargée

```

4 // Masque pour le blanc : saturation faible + luminosite elevee
5 Scalar blanc_min(0, 0, 200);
6 Scalar blanc_max(180, 50, 255);
7 Mat masque_blanc;
8 inRange(hsv, blanc_min, blanc_max, masque_blanc);
9
10 // Masque pour le jaune : teinte 15-35
11 Scalar jaune_min(15, 60, 100);
12 Scalar jaune_max(35, 255, 255);
13 Mat masque_jaune;
14 inRange(hsv, jaune_min, jaune_max, masque_jaune);
15
16 // Combinaison : conserver les pixels blancs OU jaunes
17 Mat masque_couleur;
18 bitwise_or(masque_blanc, masque_jaune, masque_couleur);

```

Listing 2 – Filtrage couleur en HSV



Masque blanc



Masque jaune



Masque combiné

FIGURE 8 – Étape 2 — Masques de couleur HSV (blanc, jaune, combiné)

5.3 Étape 3 : Détection de contours avec Canny

Principe

La luminosité change brusquement aux bords des marquages. Le flou gaussien préalable évite de détecter le bruit.

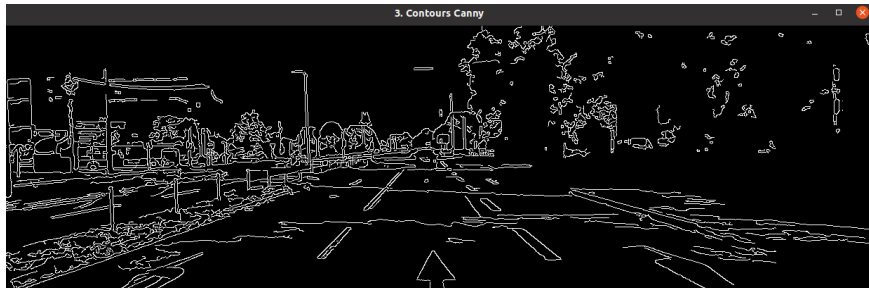
```

1 Mat gris;
2 cvtColor(image, gris, COLOR_BGR2GRAY);
3
4 Mat flou;
5 GaussianBlur(gris, flou, Size(5, 5), 0);
6

```



```
7 // seuil_bas=50, seuil_haut=150 (rapport 1:3)
8 Mat contours;
9 Canny(flou, contours, 50, 150);
10
11 // Affichage
12 namedWindow("3. Contours Canny", WINDOW_NORMAL);
13 imshow("3. Contours Canny", contours);
```

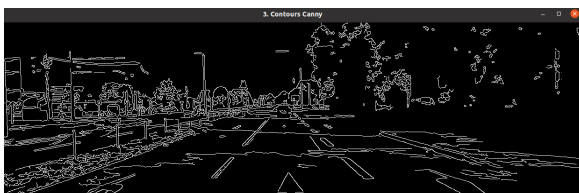
Listing 3 – Canny — fenêtre affichée : **3. Contours Canny**FIGURE 9 – Fenêtre **3. Contours Canny** — tous les contours détectés dans l'image

5.4 Étape 4 : Combinaison contours + couleur

Principe

Intersection (AND) entre contours et masque couleur : on élimine les contours parasites (arbres, bâtiments...) en ne gardant que ceux situés dans les zones blanc/jaune.

```
1 Mat combine;
2 bitwise_and(contours, contours, combine, masque_couleur);
3
4 // Affichage
5 namedWindow("4. Contours filtrés par couleur", WINDOW_NORMAL);
6 imshow("4. Contours filtrés par couleur", combine);
```

Listing 4 – Combinaison — fenêtre affichée : **4. Contours filtrés par couleur**

Avant : tous les contours



Après : contours filtrés

FIGURE 10 – Fenêtre **4. Contours filtrés par couleur** — seuls les marquages subsistent

5.5 Étape 5 : Transformation de perspective

Principe

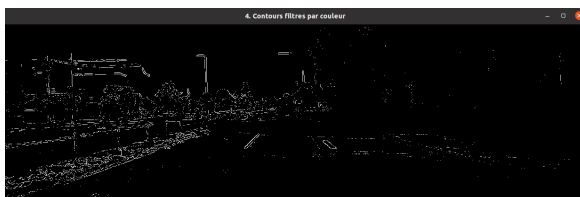
La route vue de face converge vers l'horizon. L'homographie produit une vue de dessus où les voies sont parallèles.

```

1 vector<Point2f> pts_source = {
2     Point2f(largeur * 0.15f, hauteur),
3     Point2f(largeur * 0.45f, hauteur * 0.60f),
4     Point2f(largeur * 0.55f, hauteur * 0.60f),
5     Point2f(largeur * 0.85f, hauteur)
6 };
7 vector<Point2f> pts_destination = {
8     Point2f(0, hauteur),
9     Point2f(0, 0),
10    Point2f(largeur, 0),
11    Point2f(largeur, hauteur)
12 };
13
14 Mat M = getPerspectiveTransform(pts_source, pts_destination);
15 Mat M_inv = getPerspectiveTransform(pts_destination, pts_source);
16
17 Mat vue_bird;
18 warpPerspective(combine, vue_bird, M, Size(largeur, hauteur));
19
20 // Affichage
21 namedWindow("5. Vue bird (dessus)", WINDOW_NORMAL);
22 imshow("5. Vue bird (dessus)", vue_bird);

```

Listing 5 – Perspective — fenêtre affichée : **5. Vue bird (dessus)**



Points sources sur l'image originale



Vue de dessus (bird eye)

FIGURE 11 – Fenêtre **5. Vue bird (dessus)** — voies parallèles après transformation

5.6 Étape 6 : Détection des voies (fenêtres glissantes + polynôme)

6.1 — Histogramme et bases des voies

```

1 Mat moitié_basse = image_bird(Rect(0, image_bird.rows/2,
2                                     image_bird.cols, image_bird.
3                                     rows/2));
4 Mat histogramme;
5 reduce(moitié_basse, histogramme, 0, REDUCE_SUM, CV_32S);

```

```

6 int milieu = histogramme.cols / 2;
7 Point pic_gauche, pic_droite;
8
9 minMaxLoc(histogramme(Rect(0, 0, milieu,
10      1)),
11      nullptr, nullptr, nullptr, &pic_gauche);
12 minMaxLoc(histogramme(Rect(milieu, 0, histogramme.cols - milieu,
13      1)),
14      nullptr, nullptr, nullptr, &pic_droite);
15
16 int x_base_gauche = pic_gauche.x;
17 int x_base_droite = pic_droite.x + milieu;

```

Listing 6 – Histogramme — début de la fonction detecterVoies()

6.2 — Fenêtres glissantes

```

1 int nb_fenetres = 9, marge = 100, min_pixels = 50;
2 int x_courant_gauche = x_base_gauche;
3 int x_courant_droite = x_base_droite;
4 vector<Point> pixels_voie_gauche, pixels_voie_droite;
5
6 for (int fenetre = 0; fenetre < nb_fenetres; fenetre++) {
7     int y_bas = image_bird.rows - (fenetre + 1) *
8         hauteur_fenetre;
9     int y_haut = image_bird.rows - fenetre *
10        hauteur_fenetre;
11
12     vector<Point> bons_gauche, bons_droite;
13     for (const Point& p : points_non_nuls) {
14         if (p.y >= y_bas && p.y < y_haut) {
15             if (p.x >= x_courant_gauche - marge && p.x <
16                 x_courant_gauche + marge)
17                 bons_gauche.push_back(p);
18             if (p.x >= x_courant_droite - marge && p.x <
19                 x_courant_droite + marge)
20                 bons_droite.push_back(p);
21         }
22     }
23     pixels_voie_gauche.insert(pixels_voie_gauche.end(),
24                               bons_gauche.begin(), bons_gauche.
25                               end());
26     pixels_voie_droite.insert(pixels_voie_droite.end(),
27                               bons_droite.begin(), bons_droite.
28                               end());
29
30     if (bons_gauche.size() > min_pixels) {
31         int s = 0; for (const Point& p : bons_gauche) s += p.x;
32         x_courant_gauche = s / (int)bons_gauche.size();
33     }
34     if (bons_droite.size() > min_pixels) {

```

```

29         int s = 0; for (const Point& p : bons_droite) s += p.x;
30         x_courant_droite = s / (int)bons_droite.size();
31     }
32 }

```

Listing 7 – Fenêtres glissantes

6.3 — Ajustement polynomial et génération des points

```

1 Mat A_gauche(points_gauche.size(), 3, CV_32F);
2 Mat b_gauche(points_gauche.size(), 1, CV_32F);
3 for (size_t i = 0; i < points_gauche.size(); i++) {
4     float y = points_gauche[i].x;
5     A_gauche.at<float>(i, 0) = y * y;
6     A_gauche.at<float>(i, 1) = y;
7     A_gauche.at<float>(i, 2) = 1.0f;
8     b_gauche.at<float>(i, 0) = points_gauche[i].y;
9 }
10 Mat coeffs_gauche;
11 solve(A_gauche, b_gauche, coeffs_gauche, DECOMP_SVD);
12 // coeffs_gauche = [A, B, C] => x = A*y^2 + B*y + C
13
14 // Generation d'un point par ligne de l'image
15 for (int y = 0; y < image_bird.rows; y++) {
16     float x_gauche = coeffs_gauche.at<float>(0) * y * y
17                     + coeffs_gauche.at<float>(1) * y
18                     + coeffs_gauche.at<float>(2);
19     resultat.points_gauche.push_back(Point2f(x_gauche, y));
20 }

```

Listing 8 – Moindres carrés + génération des points

5.7 Étape 7 (suite) : Visualisation des voies en vue bird

Principe

Dessiner les polynômes et remplir la zone entre les deux voies sur une image noire dans l'espace bird, avant de reprojeter.

```

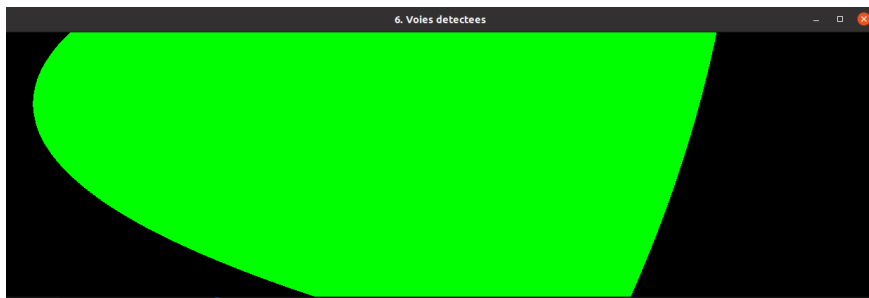
1 Mat voies_bird = Mat::zeros(vue_bird.size(), CV_8UC3);
2
3 vector<Point> pts_gauche_int, pts_droite_int;
4 for (const Point2f& p : voies.points_gauche)
5     pts_gauche_int.push_back(Point(cvRound(p.x), cvRound(p.y)));
6 for (const Point2f& p : voies.points_droite)
7     pts_droite_int.push_back(Point(cvRound(p.x), cvRound(p.y)));
8
9 polylines(voies_bird, pts_gauche_int, false, Scalar(0, 255, 0),
10          25);
11 polylines(voies_bird, pts_droite_int, false, Scalar(0, 255, 0),
12          25);

```

```

12 vector<Point> zone;
13 zone.insert(zone.end(), pts_gauche_int.begin(), pts_gauche_int.
    end());
14 zone.insert(zone.end(), pts_droite_int.rbegin(), pts_droite_int.
    rend());
15 fillPoly(voies_bird, vector<vector<Point>>{zone}, Scalar(0, 255,
    0));
16
17 // Reprojection + superposition
18 Mat voies_original;
19 warpPerspective(voies_bird, voies_original, M_inv, Size(largeur,
    hauteur));
20 Mat resultat;
21 addWeighted(image_rgb, 1.0, voies_original, 0.5, 0, resultat);
22
23 // Affichage
24 namedWindow("6. Voies detectees", WINDOW_NORMAL);
25 imshow("6. Voies detectees", voies_bird);

```

Listing 9 – Visualisation — fenêtre affichée : **6. Voies detectees**FIGURE 12 – Fenêtre **6. Voies détectées** — polynômes et zone verte en vue bird

5.8 Étape 8 : Calcul des métriques et résultat final

Principe

Convertir les coefficients pixels $\rightarrow$ mètres, calculer le rayon de courbure et le décalage latéral, puis afficher sur le résultat final.

```

1 double m_par_pixel_y = 30.0 / 720.0;
2 double m_par_pixel_x = 3.7 / 700.0;
3
4 // Conversion des coefficients en metres
5 double A_m = voies.coeffs_gauche[0] * m_par_pixel_x / (
    m_par_pixel_y * m_par_pixel_y);
6 double B_m = voies.coeffs_gauche[1] * m_par_pixel_x /
    m_par_pixel_y;
7
8 double y_eval = (vue_bird.rows - 1) * m_par_pixel_y;
9 double courbure = pow(1 + pow(2*A_m*y_eval + B_m, 2), 1.5) / fabs
    (2*A_m);

```

```

10
11 // Decalage lateral
12 double centre_voie      = (voies.points_gauche.back().x
13                             + voies.points_droite.back().x) / 2.0;
14 double decalage_m       = (vue_bird.cols/2.0 - centre_voie) *
15                             m_par_pixel_x;
16
17 putText(resultat, "Rayon: " + to_string(int(courbure)) + " m",
18         Point(50, 60), FONT_HERSHEY_SIMPLEX, 1.2, Scalar
19         (255,255,255), 3);
20 putText(resultat, "Decalage: " + to_string(fabs(decalage_m)).
21         substr(0,4)
22         + " m " + (decalage_m > 0 ? "a gauche" : "a droite"),
23         Point(50, 120), FONT_HERSHEY_SIMPLEX, 1.2, Scalar
24         (255,255,255), 3);
25
26 // Affichage
27 namedWindow("7. Resultat final", WINDOW_NORMAL);
28 imshow("7. Resultat final", resultat);
29 waitKey(0);

```

Listing 10 – Métriques + affichage — fenêtre : **7. Resultat final**FIGURE 13 – Fenêtre **7. Résultat final** — voies superposées + rayon de courbure et décalage

6. Travail à réaliser

6.1 Partie 1 : Implémentation de base

1. Téléchargez le code fourni et une image de test routière.
2. Compilez et exécutez le programme pour vérifier son fonctionnement.
3. Commentez chaque section du code pour expliquer son rôle.
4. Testez avec différentes images (droite, courbe, nuit...) et notez les résultats.

6.2 Partie 2 : Analyse et optimisation

1. **Seuils Canny** : Testez (30, 90), (50, 150), (100, 200). Observez l'impact.
2. **Plages HSV** : Les valeurs sont-elles optimales ? Proposez des ajustements nocturnes.
3. **Fenêtres glissantes** : Faites varier `nb_fenetres`, `marge`, `min_pixels`.
4. **Points source perspective** : Que se passe-t-il s'ils sont mal positionnés ?

6.3 Partie 3 : Améliorations

1. **Voie unique** : Modifiez le code si une seule voie est visible.
2. **Tracking temporel** : Proposez une méthode utilisant les frames précédentes.
3. **Interface graphique** : Ajoutez des sliders avec `createTrackbar()`.
4. **Robustesse** : Ombres, marquages usés, conditions météorologiques ?

7. Questions théoriques

1. Pourquoi utilise-t-on HSV plutôt que RGB pour le filtrage couleur ?
2. Expliquez Canny : à quoi sert le double seuillage ?
3. Différence entre transformation affine et transformation perspective ?
4. Pourquoi un polynôme du 2^e degré plutôt qu'une droite ?
5. Méthode des moindres carrés : pourquoi la décomposition SVD ?
6. Comment convertit-on des pixels en mètres ? Quelles hypothèses fait-on ?
7. Limites de cette approche ? Cas d'échec prévisibles ?

8. Annexes

8.1 Rappel sur l'espace HSV

Composante	Description
H (Teinte)	Couleur pure. Plage 0–180 dans OpenCV.
S (Saturation)	Pureté de la couleur, 0–255. Proche de 0 = gris/blanc.
V (Valeur)	Luminosité, 0–255. Proche de 0 = noir.

8.2 Formule complète du rayon de courbure

$$R(y) = \frac{[1 + (2Ay + B)^2]^{3/2}}{|2A|}$$

Évaluée en bas de l'image (y maximal), après conversion des coefficients en mètres.

8.3 Récapitulatif des 7 fenêtres affichées

N°	Titre de la fenêtre	Contenu
1	1. Image originale	Image RGB chargée depuis le disque
2	2. Masque couleur (blanc/jaune)	Masque binaire des pixels blancs et jaunes
3	3. Contours Canny	Tous les contours détectés dans l'image
4	4. Contours filtrés par couleur	Contours gardés uniquement dans les zones blanc/jaune
5	5. Vue bird (dessus)	Image transformée en vue de dessus
6	6. Voies détectées	Polynômes + zone verte en vue bird
7	7. Résultat final	Voies superposées + rayon de courbure + décalage

8.4 Commandes de compilation

```

1 g++ -o detection_voies detection_voies.cpp \
2   'pkg-config --cflags --libs opencv4'
3 ./detection_voies

```

Listing 11 – Compilation et exécution avec OpenCV 4